

Full Stack Web Development Lab

Week 6 – Record

Aim

To write a Node.js program that demonstrates the use of core (built-in) Node.js modules — **os**, **path**, **dns**, **net**, and **readline** — to display system information, parse a file path entered by the user, resolve a domain name to its IP address, and run a simple TCP server.

Description

This lab moves from browser-based JavaScript to server-side JavaScript using **Node.js**. The program (**app.js**) requires five core modules and uses them one after another: the **os** module prints the platform, CPU architecture, CPU model/core count, and total/free memory of the machine. The **readline** module creates an interface to read input from the terminal, first asking the user for a file path, which is then broken down using the **path** module (**dirname**, **basename**, **extname**, **normalize**). Next, the program asks for a domain name and uses **dns.lookup()** to resolve it to an IP address. Finally, the **net** module is used to start a TCP server (**net.createServer()**) that listens on port 5000, logs when a client connects (with its remote address and port), sends a welcome message, and logs when the client disconnects. The server can be tested using a TCP client such as **telnet localhost 5000**.

Program

app.js

```
const os = require('os');
const path = require('path');
const dns = require('dns');
const net = require('net');
const readline = require('readline');

const rl = readline.createInterface({ input: process.stdin, output: process.stdout });

// ----- OS MODULE -----
console.log('--- OS Info ---');
console.log('Platform:', os.platform());
console.log('CPU Architecture:', os.arch());
console.log('CPU Info:', os.cpus()[0].model, `(${os.cpus().length} cores)`);
console.log('Total Memory:', (os.totalmem() / (1024 ** 3)).toFixed(2), 'GB');
console.log('Free Memory:', (os.freemem() / (1024 ** 3)).toFixed(2), 'GB');
console.log();

// ----- PATH MODULE -----
rl.question('Enter a file path: ', (filePath) => {
  console.log('\n--- Path Info ---');
  console.log('Directory Name:', path.dirname(filePath));
  console.log('File Name:', path.basename(filePath));
  console.log('Extension:', path.extname(filePath));
  console.log('Normalized Path:', path.normalize(filePath));
  console.log();
});

// ----- DNS MODULE -----
rl.question('Enter a domain name: ', (domain) => {
  dns.lookup(domain, (err, address) => {
    console.log('\n--- DNS Info ---');
    if (err) {
      console.log('Error resolving domain:', err.message);
    } else {
      console.log(`IP address of ${domain}:`, address);
    }
  });
  rl.close();
});
```

```

        // ----- NET MODULE -----
        startServer();
    });
});
});

// ----- NET MODULE (TCP Server) -----
function startServer() {
    const server = net.createServer((socket) => {
        console.log('\n--- Net Info ---');
        console.log('Client connected:', socket.remoteAddress + ':' + socket.remotePort);
        socket.write('Welcome to the server!\n');

        socket.on('end', () => {
            console.log('Client disconnected');
        });
    });

    const PORT = 5000;
    server.listen(PORT, () => {
        console.log(`\nTCP server running on port ${PORT}`);
        console.log('Connect using: telnet localhost 5000');
    });
}

```

Details about the Program

Where Saved	The file is saved as app.js inside the project folder FL_LAB_ANITS/fs_week_6/ in the GitHub repository raviteja-codehub/FL_LAB_ANITS .
Where Executed	Unlike the earlier weeks, this program does not run inside a web browser. It is executed on the Node.js runtime from a terminal / command prompt (e.g. PowerShell), since it uses Node's built-in server-side modules (os, path, dns, net, readline) which are not available in browser JavaScript.
How Executed	1. Open a terminal (e.g. PowerShell) and navigate to the project folder using cd fs_week_6. 2. Run the program with node app.js. Node.js first prints the OS information automatically. 3. The program then prompts "Enter a file path:" — typing a path (e.g. C:\Users\pasup\Documents\report.pdf) and pressing Enter prints its directory, file name, extension, and normalized form. 4. It then prompts "Enter a domain name:" — typing a domain (e.g. raviteja.com) resolves and prints its IP address using DNS. 5. Finally, it starts a TCP server on port 5000. Connecting to it from another terminal using telnet localhost 5000 (or any TCP client) logs the client's connection and disconnection on the server's console.

Output

Running **node app.js** in the terminal, entering a file path and a domain name, and then connecting/disconnecting TCP clients produces the following console output:

```
PS C:\Users\pasup\OneDrive\Desktop\FS LAB> cd fs_week_6
PS C:\Users\pasup\OneDrive\Desktop\FS LAB\fs_week_6> node app.js
--- OS Info ---
Platform: win32
CPU Architecture: x64
CPU Info: 13th Gen Intel(R) Core(TM) i7-13620H (16 cores)
Total Memory: 15.70 GB
Free Memory: 3.88 GB

Enter a file path: C:\Users\pasup\Documents\report.pdf

--- Path Info ---
Directory Name: C:\Users\pasup\Documents
File Name: report.pdf
Extension: .pdf
Normalized Path: C:\Users\pasup\Documents\report.pdf

Enter a domain name: raviteja.com

--- DNS Info ---
IP address of raviteja.com: 172.239.194.117

TCP server running on port 5000
Connect using: telnet localhost 5000

--- Net Info ---
Client connected: ::1:59509

--- Net Info ---
Client connected: ::1:52567
Client disconnected
█
```